Lab-19.3

Name: D. Suraj
2303A53006
Batch:46

Lab 19 – Code Translation: Converting Between Programming Languages

Task 1 – Python to C++ Conversion

Task:
Translate a simple Python class into C++ using AI assistance.

Instructions:

- Prompt AI to convert a Python class representing a Student with attributes and methods into equivalent C++ code
- Ensure proper handling of:
  - Constructors
  - Data types
  - Access specifiers
- Compile and run both versions to verify output consistency

Expected Output:
An equivalent working C++ class translated from Python.

Python Implementation:

First, define the source Python class. This class uses a constructor (__init__) and a simple method to display data.

C++ Implementation:

The AI-generated C++ code handles memory and type safety explicitly, which is a key difference between the two languages.

Prompt:
"Convert the following Python Student class into C++. Ensure you use a proper constructor, map data types correctly (string, int, double), and use access specifiers (private for attributes, public for methods). Include a main function to demonstrate the output."

Code in python:

```python
# Python Source Code
class Student:
    def __init__(self, name, roll_number, gpa):
        self.name = name            # String attribute
        self.roll_number = roll_number  # Integer attribute
        self.gpa = gpa              # Float attribute

    def display_details(self):
        print(f"Student Name: {self.name}")
        print(f"Roll Number: {self.roll_number}")
        print(f"GPA: {self.gpa}")

# Testing the class
s1 = Student("Aryan Sharma", 101, 8.5)
s1.display_details()
```

Output:

```
Student Name: Aryan Sharma
Roll Number: 101
GPA: 8.5


...Program finished with exit code 0
Press ENTER to exit console.
```

Code in C++:

```cpp
#include <iostream>
#include <string>
class Student {
private:
    // Attributes (Access Specifier: private)
    std::string name;
    int roll_number;
    double gpa;
public:
    // Constructor (Proper handling of initialization)
    Student(std::string n, int r, double g) {
        name = n;
        roll_number = r;
        gpa = g;
    }
    // Method to display details (Access Specifier: public)
    void display_details() {
        std::cout << "Student Name: " << name << std::endl;
        std::cout << "Roll Number: " << roll_number << std::endl;
        std::cout << "GPA: " << gpa << std::endl;
    }
};
int main() {
    // Instantiating the class
    Student s1("Aryan Sharma", 101, 8.5);
    s1.display_details();
    return 0;
}
```

Output:

```
Student Name: Aryan Sharma
Roll Number: 101
GPA: 8.5


...Program finished with exit code 0
Press ENTER to exit console.
```

Explanation & Comparison:

In your report, include these observations regarding the translation:

- Access Specifiers: Python does not have strict access control (everything is public by default), whereas C++ requires explicit private and public blocks to enforce encapsulation.
- Data Types: Python is dynamically typed; C++ is statically typed. We had to explicitly define std::string, int, and double.
- Constructors: In Python, the constructor is __init__. In C++, the constructor has the same name as the class and no return type.
- Syntax: Python uses indentation to define scope, while C++ uses curly braces {} and semicolons ; .

Task 2 – Java to Python Function Conversion

Task:
Convert a Java method that checks whether a number is prime into Python.

Instructions:

- Ask AI to translate a Java isPrime() method into Python
- Test the function with multiple inputs
- Ensure the Python version follows Pythonic syntax and logic

Expected Output:
A correct Python function equivalent to the original Java prime-checking method.

Java Implementation
This is the standard Java method using a for loop and the Math.sqrt() function for efficiency.

Python Implementation
The AI simplifies the syntax by removing curly braces and using the range object.

Prompt:
"Translate this Java isPrime method into Python. Ensure the resulting code is 'Pythonic'—use the math module for the square root, use a range-based loop, and include a list of test cases to verify the output for multiple numbers."

Code :

```java
// Java Source Code
public class PrimeTask {
    public static boolean isPrime(int n) {
        if (n <= 1) return false;
        for (int i = 2; i <= Math.sqrt(n); i++) {
            if (n % i == 0) return false;
        }
        return true;
    }

    public static void main(String[] args) {
        System.out.println("7 is prime: " + isPrime(7));
    }
}
```

```python
import math

def is_prime(n):
    """
    Checks if a number is prime.
    Following Pythonic logic: handles edge cases and uses an optimized loop.
    """
    if n <= 1:
        return False

    # We check up to the square root of n for efficiency
    for i in range(2, int(math.sqrt(n)) + 1):
        if n % i == 0:
            return False
    return True

# --- Testing with multiple inputs ---
test_numbers = [2, 4, 7, 15, 19, 25, 29]
print("Number | Is Prime?")
print("-" * 18)
for num in test_numbers:
    print(f"{num:6} | {is_prime(num)}")
```

Output:

```
Number | Is Prime?
------------------
     2 | True
     4 | False
     7 | True
    15 | False
    19 | True
    25 | False
    29 | True


...Program finished with exit code 0
Press ENTER to exit console.
```

Explanation & Comparison:

In your report, highlight these key differences found during the translation:

- Type Declaration: Java requires int n and boolean return types. Python is dynamically typed, so no types are declared.
- Loop Syntax: Java uses for (int i = 2; i <= limit; i++). Python uses for i in range(2, limit + 1), which is more readable but requires adding +1 because the stop value is exclusive.
- Booleans: Java uses lowercase true/false, while Python uses capitalized True/False.
- Library Import: Python requires import math to use math.sqrt(), whereas Java includes it in the standard java.lang package.

Task 3 – Pseudocode to Python Implementation

Task:

Translate a given pseudocode algorithm into Python using AI.

Instructions:

- Provide AI with pseudocode for sorting numbers (e.g., bubble sort or selection sort)
- Ask AI to convert it into executable Python code
- Validate correctness using sample input lists

Expected Output:

A working Python program that correctly implements the pseudocode logic.

Pseudocode (Input)

Pseudocode is a simplified, language-independent way to write algorithms. You will provide this to the AI:

Python Implementation

The AI translates the abstract "FOR" loops and "SWAP" commands into valid Python syntax.

Prompt:

"I am providing pseudocode for a Selection Sort algorithm. Please translate this into executable Python code. Ensure you use descriptive variable names, include comments explaining the logic, and provide a sample list of numbers to test and validate the sorting results."

Code:

```
PROCEDURE selectionSort(list)
    n = length(list)
    FOR i FROM 0 TO n-1
        min_index = i
        FOR j FROM i+1 TO n-1
            IF list[j] < list[min_index] THEN
                min_index = j
            END IF
        END FOR
        SWAP list[i] with list[min_index]
    END FOR
END PROCEDURE
```

```python
def selection_sort(arr):
    """
    Implementation of Selection Sort translated from pseudocode.
    """
    n = len(arr)

    # Outer loop to move the boundary of the unsorted subarray
    for i in range(n):
        # Find the minimum element in the remaining unsorted array
        min_index = i
        for j in range(i + 1, n):
            if arr[j] < arr[min_index]:
                min_index = j

        # Swap the found minimum element with the first element
        # Pythonic swap: no temporary variable needed
        arr[i], arr[min_index] = arr[min_index], arr[i]

    return arr

# --- Validation with Sample Input ---
sample_data = [64, 25, 12, 22, 11]
print("Original List:", sample_data)

sorted_data = selection_sort(sample_data.copy())
print("Sorted List:  ", sorted_data)

# Verification check
if sorted_data == sorted(sample_data):
    print("\nResult: SUCCESS! The algorithm correctly sorted the list.")
else:
    print("\nResult: FAILURE. The sorting logic is incorrect.")
```

Output:

```
Original List: [64, 25, 12, 22, 11]
Sorted List:   [11, 12, 22, 25, 64]

Result: SUCCESS! The algorithm correctly sorted the list.


...Program finished with exit code 0
Press ENTER to exit console.
```

Explanation & Logic Mapping

In your report, map the pseudocode commands to the Python code:

- length(list) -> rightarrow -> became len(arr).
- FOR i FROM 0 TO n-1 -> rightarrow -> became for i in range(n).
- SWAP list[i] with list[min_index] -> rightarrow -> Python handles this elegantly with tuple unpacking: a, b = b, a.
- END FOR / END IF -> rightarrow -> In Python, these are replaced by indentation levels, making the code cleaner.

Task 4 – SQL to Pandas Query

Task:

Translate a SQL query into a Pandas DataFrame operation in Python.

Instructions:

- Provide AI with a SQL query (e.g., SELECT name, salary FROM employees WHERE salary > 50000).
- Ask AI to generate equivalent Pandas code.
- Test the generated code on a sample DataFrame.

Expected Output:

• Equivalent Pandas query that retrieves the correct subset of data.

Prompt:

"Translate the following SQL query into Python Pandas code: 'SELECT name, salary FROM employees WHERE salary > 50000'. First, create a sample DataFrame called 'employees' with columns 'id', 'name', 'salary', and 'department'. Then, show the equivalent Pandas operation to filter and select the specific columns. Explain the mapping between the SQL keywords and Pandas methods."

SQL Query (Input)

We will use a standard filtering and column selection query:

```sql
SELECT name, salary
FROM employees
WHERE salary > 50000;
```

Pandas Implementation

The AI-generated code uses boolean indexing, which is the most common way to filter data in Pandas.

```python
import pandas as pd

# Step 1: Create a sample DataFrame
data = {
    'id': [1, 2, 3, 4, 5],
    'name': ['Amit', 'Priya', 'Rahul', 'Sonia', 'Vikram'],
    'salary': [45000, 60000, 52000, 30000, 75000],
    'department': ['HR', 'IT', 'IT', 'Sales', 'Engineering']
}

employees = pd.DataFrame(data)

# Step 2: Equivalent Pandas Query
# SQL: SELECT name, salary FROM employees WHERE salary > 50000
result = employees[employees['salary'] > 50000][['name', 'salary']]

# Step 3: Display results
print("Original DataFrame:")
print(employees)
print("\nFiltered Result (Salary > 50000):")
print(result)
```

Output:

```
Original DataFrame:
   id    name  salary   department
0   1    Amit   45000           HR
1   2   Priya   60000           IT
2   3   Rahul   52000           IT
3   4   Sonia   30000        Sales
4   5  Vikram   75000  Engineering

Filtered Result (Salary > 50000):
     name  salary
1   Priya   60000
2   Rahul   52000
4  Vikram   75000


...Program finished with exit code 0
Press ENTER to exit console.
```

SQL to Pandas Mapping Logic

For your report, use this table to document how the translation works:

SQL Keyword	Pandas Equivalent	Explanation
FROM employees	employees	Refers to the DataFrame object name.
WHERE salary > 50000	employees['salary'] > 50000	Boolean indexing to filter rows.
SELECT name, salary	[['name', 'salary']]	Subsetting the DataFrame by column names.

Explanation:

SQL to Pandas (Data Manipulation)

- **Declarative vs. Imperative:** SQL is **declarative** (you tell the computer *what* you want). Pandas is **imperative/procedural** (you tell the computer *how* to get it).
- **Filtering:** The WHERE clause in SQL is translated into **Boolean Indexing** in Pandas. The AI creates a mask (e.g., df['salary'] > 50000) that returns only the rows where the condition is True.
- **Projection:** The SELECT part is handled by passing a list of column names [['name', 'salary']] to the DataFrame.

Task 5 – Real-Time Application: Algorithm Translation Across Languages

Scenario:

A company maintains algorithms written in different programming languages.

Examples:

- Converting a Python searching algorithm into C
- Translating a Java sorting program into JavaScript

Instructions:

- Use AI to translate a selected algorithm between two programming languages
- Execute and test both versions
- Document translation challenges such as:
    - Syntax differences
    - Library support
    - Memory management

Expected Output:

Equivalent, tested code implementations in two different programming languages.

Prompt:

"Translate this Python Binary Search algorithm into C. In the C version, ensure you handle array sizing manually (since C doesn't have a len() function for pointers) and use appropriate integer types. Include a main function that initializes a sorted array and prints the result using printf."

Algorithm (Python)

Python's version is concise and handles list slicing and memory automatically.

main.py

```python
# Python Implementation of Binary Search
def binary_search(arr, target):
    low = 0
    high = len(arr) - 1

    while low <= high:
        mid = (low + high) // 2
        if arr[mid] == target:
            return mid
        elif arr[mid] < target:
            low = mid + 1
        else:
            high = mid - 1
    return -1

# Test
arr = [2, 3, 4, 10, 40]
print(f"Element found at index: {binary_search(arr, 10)}")
```

input

```
Element found at index: 3


...Program finished with exit code 0
Press ENTER to exit console.
```

Implementation (C)

The C version requires manual array size calculation and explicit header inclusions.

main.c

```c
#include <stdio.h>
// C Implementation of Binary Search
int binarySearch(int arr[], int size, int target) {
    int low = 0;
    int high = size - 1;

    while (low <= high) {
        int mid = low + (high - low) / 2; // Avoids potential overflow

        if (arr[mid] == target)
            return mid;
        if (arr[mid] < target)
            low = mid + 1;
        else
            high = mid - 1;
    }
    return -1;
}
int main() {
    int arr[] = {2, 3, 4, 10, 40};
    int n = sizeof(arr) / sizeof(arr[0]); // Manual size calculation
    int target = 10;
    int result = binarySearch(arr, n, target);
    if(result != -1)
        printf("Element found at index: %d\n", result);
    else
        printf("Element not found\n");
    return 0;
}
```

input

```
Element found at index: 3


...Program finished with exit code 0
```

Explanation:

Algorithm Translation (C vs. Python)

- **Manual vs. Automatic:** This task highlights the biggest challenge: **Abstraction levels**.
- **The "Size" Problem:** In Python, a list "knows" its size (len(arr)). In C, an array is just a pointer to a memory location. The AI must add a size parameter to the function because C cannot determine the length of a passed array automatically.
- **Safety:** C is "closer to the metal." The AI uses low + (high - low) / 2 for the midpoint to prevent **Integer Overflow**, a common bug in low-level languages that Python handles automatically by using arbitrary-precision integers.